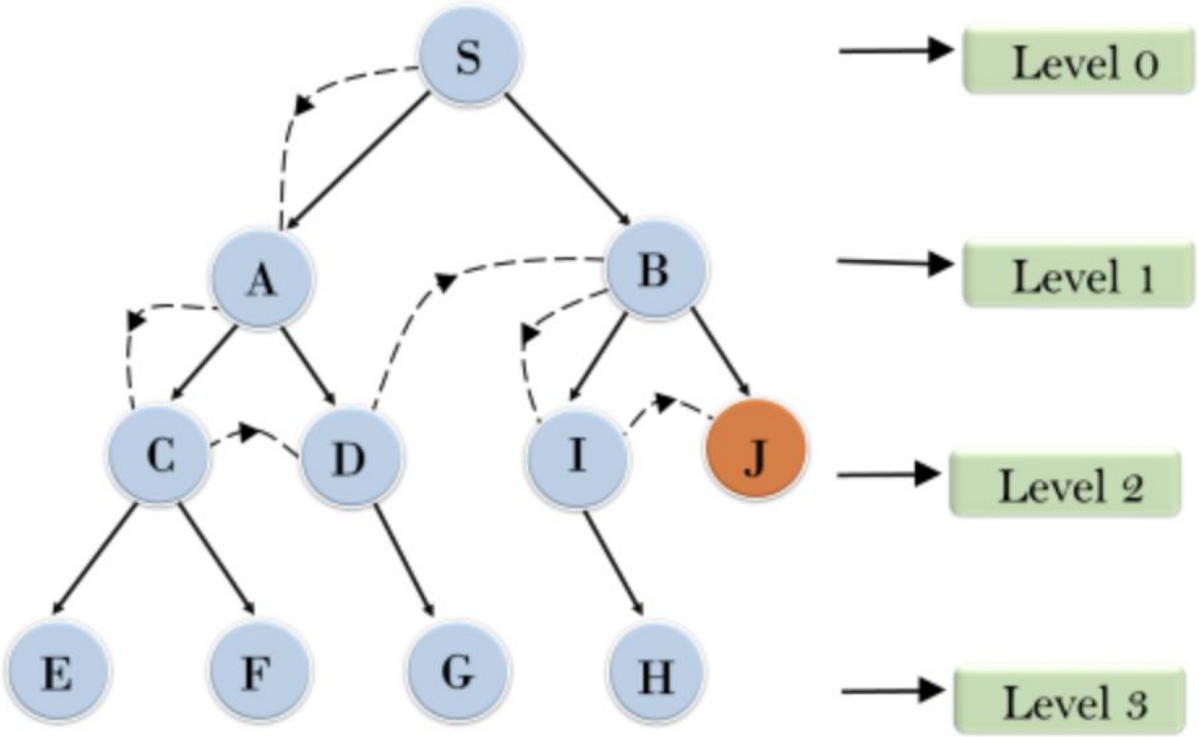
Depth-Limited Search

Depth-Limited Search (DLS) is a variation of the Depth-First Search (DFS) algorithm that imposes a maximum depth limit to prevent the search from going too deep into the search space. This approach is particularly useful in scenarios where infinite paths may exist, such as in certain graph or tree structures.

Key Characteristics of Depth-Limited Search:

1. **Depth Limit**: DLS introduces a parameter that specifies the maximum depth to which the search can explore. If the depth limit is reached, the search backtracks without exploring further.
2. **Controlled Exploration**: By limiting the depth, DLS avoids the risk of running indefinitely in deep or infinite structures, making it more efficient in certain contexts.
3. **Stack-Based**: Like DFS, DLS uses a stack (either explicitly or through recursion) to manage the nodes being explored.

Example



DLS Algorithm Steps

1. **Initialize**:
    - Create a stack to manage nodes to explore.
    - Push the starting node onto the stack along with its depth (initially 0).
    - Define a maximum depth limit.
2. **Process Nodes**:
    - While the stack is not empty:
        - Pop a node and its depth from the stack.
        - If the depth is equal to the limit, backtrack (do not explore further).
        - If the node is the goal node, return it as the solution.
        - Otherwise, mark the node as visited.
        - Push all unvisited neighbors of the current node onto the stack, incrementing their depth by one.
3. **Terminate**:
    - If the stack becomes empty without finding the goal node, report that no solution exists within the depth limit.

Algorithm

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
      cutoff_occurred? ← false
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          result ← RECURSIVE-DLS(child, problem, limit − 1)
          if result = cutoff then cutoff_occurred? ← true
          else if result ≠ failure then return result
      if cutoff_occurred? then return cutoff else return failure
```

Time Complexity

Depth-Limited Search (DLS) is a variation of Depth-First Search (DFS) where the search is limited to a specified depth ***l***. The time complexity of DLS depends on the branching factor ***b*** (the average number of child nodes for each node) and the depth limit ***l***.

1. Understand the Tree Structure:

In DLS, the search tree can be visualized as a complete tree where:

- **Root node:** At depth 0.
- **Level 1:** Contains ***b*** nodes.
- **Level 2:** Contains b^2 nodes.
- ...
- **Level l:** Contains b^l nodes.

The total number of nodes in a tree up to depth ***l*** is the sum of the nodes at each level:

$$\textbf{Total nodes} = 1 + b + b^2 + \cdots + b^l$$

2. Calculate the Sum of the Series:

This sum represents the total number of nodes that the DLS algorithm will potentially explore. The sum of a geometric series can be calculated using the formula:

$$S = \frac{b^{l+1} - 1}{b - 1}$$

3. Simplifying for Large l

For large ***l***, the term b^{l+1} dominates the sum ***S***, making the smaller terms negligible. This means the sum ***S*** can be approximated by just considering the largest term in the series, which is b^l:

$$S \approx \frac{b^{l+1} - 1}{b - 1} \approx \frac{b^{l+1}}{b - 1}$$

Now, to derive the time complexity, we focus on the dominant term that contributes to the overall growth of the function. The b^{l+1} term is the most significant, and since it dominates, the time complexity is approximated as:

$$O(b^{l+1}) \text{ where } b^{l+1} = b \cdot b^l = O(b^l)$$

Why l+1 Becomes l:

- **Mathematical Insight:** When considering time complexity, constants and lower-order terms are typically ignored because they do not affect the asymptotic behavior. In the expression b^{l+1}, the base ***b*** is constant, so increasing the exponent by 1 (which is what happens when you go from $l-1$ to ***l***) does not fundamentally change the asymptotic growth rate. Thus, both b^{l+1} and b^l grow at a similar exponential rate.
- **Simplification:** We drop the constant factor and focus on the term that defines the exponential growth, which is b^l. Hence, the time complexity is simplified to $O(b^l)$.

Completeness of DLS

- **Scenario 1: The Goal is Within the Depth Limit l:**
  - If the goal node is at a depth ***d ≤ l***, Depth-Limited Search (DLS) will find the goal. This is because DLS will explore all nodes up to the depth limit ***l***. Since DLS systematically explores every possible path within the depth limit, it is guaranteed to reach the goal if it exists within that limit.
- **Scenario 2: The Goal is Deeper than the Depth Limit l:**
  - If the goal node is at a depth ***d > l***, DLS will not find the goal because it will not explore nodes beyond depth ***l***. Therefore, in this case, DLS is not complete because it fails to find the solution that lies beyond its depth limit.

Mathematical Expression of Completeness:

Let ***d*** be the depth at which the goal node ***g*** is located. DLS is complete if:

$$d \leq l$$

where ***l*** is the depth limit. If ***d>l***, DLS is incomplete.

Conclusion on Completeness:

- **Complete when d ≤ l :** If the goal lies within the depth limit, DLS is complete.
- **Incomplete when d > l:** If the goal lies beyond the depth limit, DLS is not complete.

Optimality of DLS

- DLS explores nodes at increasing depths up to the limit ***l***, similar to Depth-First Search (DFS). However, DLS is not guaranteed to find the shortest path if there are multiple paths to the goal.
- DLS might find a solution at a greater depth before discovering a shallower path because it explores deeply before considering alternative, potentially shorter paths at shallower levels.

Mathematical Proof of Non-Optimality:

Let there be two paths to the goal ***g***:

1. **Path 1:** Length d_1
2. **Path 2:** Length d_2

Assume $d_1 < d_2$

- If DLS explores Path 2 first because the nodes along Path 2 are explored before those along Path 1, it will find the solution at depth d_2 before discovering the shorter path of length d_1.
- Therefore, DLS can return a solution that is not optimal because it does not consider the depth or cost of the paths it explores.

Conclusion on Optimality:

- **Not Optimal:** DLS is not guaranteed to find the shortest path, as it may find a solution at a greater depth before exploring all shallower paths.

DLS Performance Measure

Time Complexity: The time complexity of DLS is $O(b^l)$, where b is the branching factor and l is the depth limit. This can be more efficient than standard DFS if the limit is set appropriately.

Space Complexity: The space complexity remains $O(l)$, where l is the depth limit, as only nodes up to that depth need to be stored in memory.

Completeness: DLS is complete only if the solution lies within the specified depth limit. If the solution is deeper than the limit, DLS will fail to find it.

Optimality: DLS does not guarantee an optimal solution, as it may not explore all possible paths if they exceed the depth limit.

DLS Applications

Search Space Pruning: DLS can be used in optimization problems where certain solutions are known to be infeasible beyond a certain depth. By applying depth limits, DLS can prune large portions of the search space early, improving efficiency.

Hierarchical Problem Solving: In hierarchical problem-solving environments, DLS can be employed to navigate through different levels of abstraction. For example, it can be used to explore various levels of a decision tree where only a limited number of decisions need to be considered at once.

Algorithm Testing and Validation: DLS can be useful in testing algorithms that operate in recursive environments by restricting the depth of recursion, allowing developers to verify behavior and performance without risking stack overflow or excessive computation.

Interactive Game Development: In interactive gaming, DLS can help limit the depth of AI decision-making processes to maintain responsiveness. This ensures that AI characters make timely decisions without evaluating all potential future states.

State Space Exploration: In fields like robotics or automated planning, DLS can be used to explore state spaces where certain states may require excessive resources to evaluate. By limiting the depth, robots can focus on immediate and feasible actions while ignoring less relevant deep states.

Limitations of Depth-Limited Search (DLS)

1. Risk of Incompleteness

Limitation:

- **Missing the Solution:** DLS is not guaranteed to find a solution if the goal node is deeper than the specified depth limit. If the solution exists beyond the depth limit, DLS will terminate without finding the goal, leading to incompleteness.

Scenarios Where This is Inefficient:

- **Deep Search Spaces:** In problems where the solution may exist at an unknown or deep level, such as deep puzzle-solving or long-path routing problems, setting an arbitrary depth limit may cause DLS to miss the solution entirely.
- **Unknown Solution Depth:** In situations where the depth of the solution is not known in advance, DLS might be inefficient because it might miss the goal simply due to an inadequate depth limit.

2. Sensitivity to Depth Limit Selection

Limitation:

- **Optimal Depth Limit is Unknown:** The efficiency of DLS heavily depends on choosing an appropriate depth limit. If the limit is set too low, the algorithm may fail to find the solution even if it exists. If the limit is set too high, the search may explore unnecessary paths, reducing efficiency.

Scenarios Where This is Inefficient:

- **Dynamic or Complex Environments:** In environments like AI decision-making or dynamic systems where the depth of the goal can vary, selecting an appropriate depth limit can be challenging, leading to inefficiencies.
- **Exploratory Problems:** In problems where the search space is complex or poorly understood, and the optimal depth limit is unclear, DLS might waste time exploring irrelevant paths or fail to reach the goal.

3. Inefficiency in Large and Wide Search Trees

Limitation:

- **High Branching Factor:** In search spaces with a high branching factor, where each node has many children, DLS can become inefficient. The algorithm might spend a considerable amount of time exploring wide portions of the tree at shallow depths without making significant progress toward deeper nodes where the goal might lie.

Scenarios Where This is Inefficient:

- **Large Search Trees:** In large decision trees, such as those found in game theory or combinatorial optimization problems, DLS might explore many unnecessary branches at shallow levels, leading to inefficient use of time and resources.
- **Wide Networks:** In network routing or communication systems with many possible paths, DLS might be inefficient because it cannot focus on deeper nodes where the goal might be located.

4. Lack of Optimality

Limitation:

- **Non-Optimal Solutions:** DLS does not guarantee that it will find the optimal solution, even if it finds the goal within the depth limit. The first solution found may not be the shortest or least costly path, as DLS does not prioritize paths based on cost or distance.

Scenarios Where This is Inefficient:

- **Pathfinding in Weighted Graphs:** In scenarios where finding the least-cost path is critical, such as in transportation or logistics planning, DLS may return a suboptimal path, making it less suitable for such applications.
- **Optimization Problems:** In problems like resource allocation or project scheduling, where the optimal solution is essential, DLS may provide a solution, but it might not be the best one available.

5. Potential for Redundant Exploration

Limitation:

- **Repeated Node Visits:** In DLS, the same nodes can be revisited multiple times across different recursive calls if the depth limit is not carefully managed. This redundancy can lead to wasted computational effort.

Scenarios Where This is Inefficient:

- **Cyclic Graphs:** In cyclic graphs, where nodes can be revisited through different paths, DLS might waste time exploring the same nodes repeatedly, leading to inefficiency.
- **Large Repetitive Structures:** In search spaces with repetitive structures or patterns, such as certain types of mazes or puzzles, DLS may revisit the same states multiple times, slowing down the search process.

6. Memory Usage Concerns in Deep Searches

Limitation:

- **Stack Overflow Risk:** Although DLS is more memory-efficient than BFS, it still relies on recursion or a stack to manage the depth-first exploration. In very deep searches, the risk of stack overflow becomes significant, particularly in environments with limited memory resources.

Scenarios Where This is Inefficient:

- **Deep Recursion:** In problems requiring deep exploration, such as complex puzzle-solving or long-term planning, DLS can run into stack overflow issues, especially on systems with limited stack size.
- **Embedded Systems:** In memory-constrained environments like embedded systems or IoT devices, DLS might be inefficient due to its reliance on recursive depth-first exploration, which can exhaust available memory.

7. Limited Applicability to Real-Time Systems

Limitation:

- **Time Constraints:** DLS is not well-suited for real-time systems where decisions need to be made within strict time limits. The algorithm's reliance on exploring nodes up to a certain depth can lead to unpredictable delays, especially if the depth limit is not well-calibrated.

Scenarios Where This is Inefficient:

- **Real-Time Decision-Making:** In applications like robotics or real-time game AI, where timely responses are critical, DLS may introduce delays due to its depth-limited exploration, making it less suitable for real-time environments.
- **Interactive Systems:** In interactive applications where the system needs to respond quickly to user input, DLS might be inefficient if the depth limit causes unnecessary delays or fails to find a timely solution.

8. Incomplete in Infinite Graphs

Limitation:

- **Incompleteness in Infinite Spaces:** DLS is incomplete in infinite graphs or search spaces. If the graph has an infinite structure and the solution lies beyond the depth limit, DLS will not find it, making the algorithm unsuitable for such problems.

Scenarios Where This is Inefficient:

- **Infinite State Spaces:** In problems like certain types of puzzles or theoretical computer science problems where the search space is infinite, DLS might terminate prematurely, failing to find a solution that exists at a greater depth.
- **Open-Ended Problems:** In open-ended AI tasks, where the search space is not well-defined or bounded, DLS may struggle to find solutions if they are located beyond the arbitrarily set depth limit.

Iterative Deepening Search

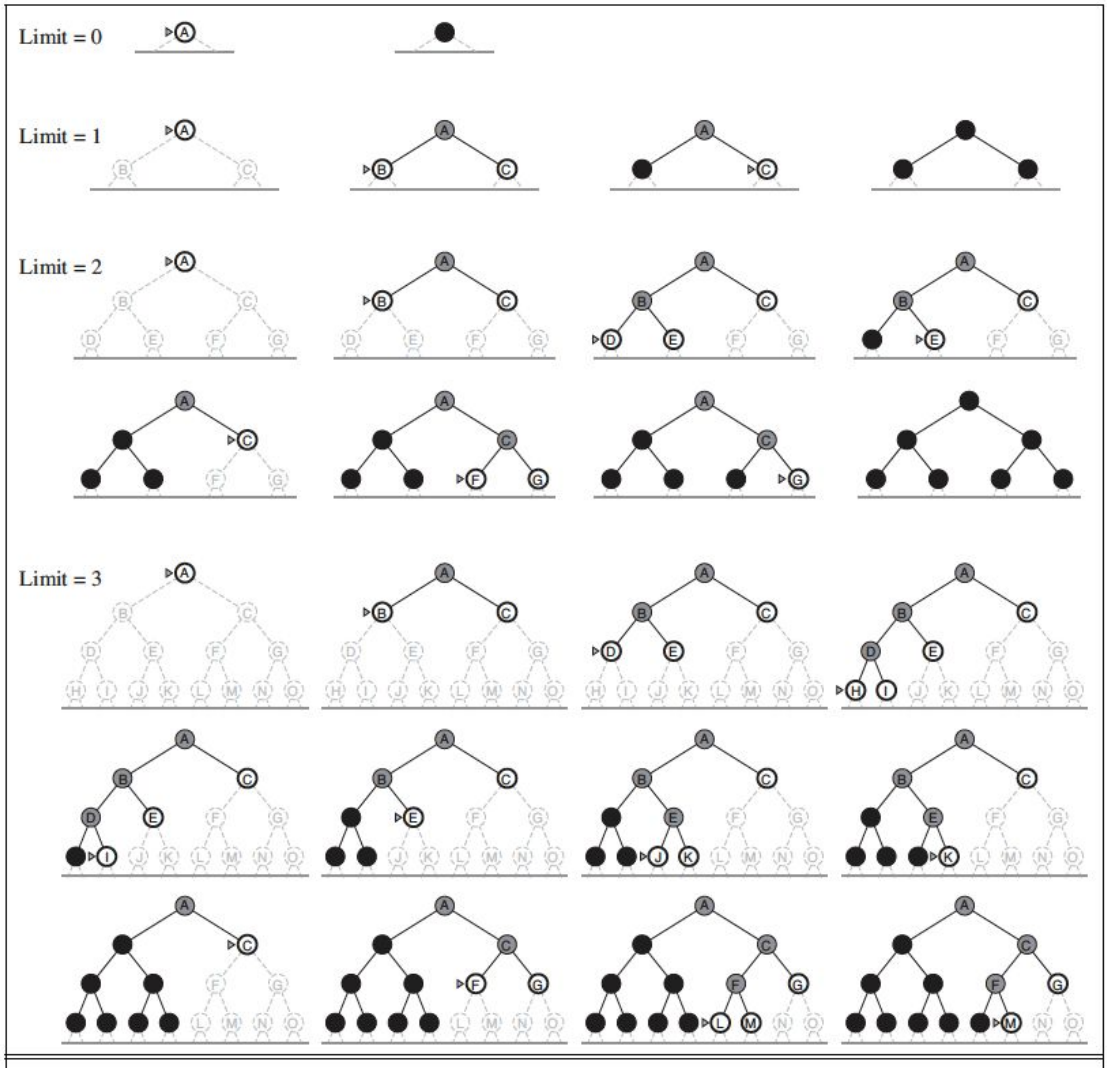
Iterative Deepening Search (IDS) is a search algorithm that combines the benefits of Depth-First Search (DFS) and Breadth-First Search (BFS). It performs a series of depth-limited searches, incrementally increasing the depth limit with each iteration until a solution is found. This approach is particularly useful for searching large or infinite state spaces while ensuring optimality and completeness.

Key Characteristics of Iterative Deepening Search:

1. **Combination of DFS and BFS**: IDS utilizes the depth-limited search methodology, effectively exploring the search space like DFS but doing so iteratively with increasing depth limits.
2. **Memory Efficiency**: IDS uses much less memory than BFS because it only needs to store the nodes along the current path and the depth limit, making it suitable for large state spaces.

Example

Four iterations of iterative deepening search on a binary tree.



IDS Algorithm Steps

Initialize:

- Set the initial depth limit (usually starting at 0).
- Create a loop that will run until a solution is found.

Depth-Limited Search:

- Perform a depth-limited search for the current depth limit:
  - Start from the initial node and explore its children.
  - If the depth of the current node equals the limit, stop exploring further down that branch.
  - If the current node is the goal node, return it as the solution.
  - Otherwise, push unvisited neighbors onto the stack with their depth incremented by one.

Increment Depth Limit:

- If the depth-limited search fails to find a solution, increment the depth limit by one and repeat the process.

Terminate:

- Continue this process until a solution is found or the search space is exhausted.

Proof of Optimality

Argument for Optimality in Unweighted Graphs:

- **Shallowest Goal First:** In an unweighted graph, the cost of reaching a node is proportional to the depth of the node. Therefore, the optimal solution is the one found at the shallowest depth.
- **Mechanism:** IDS performs a complete search at each depth level before moving to the next level. This means that IDS will explore all nodes at depth ***d*** before exploring any nodes at depth ***d+1***.
- **Guarantee:** Since IDS systematically searches at increasing depths and finds the goal at the shallowest possible depth, it is guaranteed to find the optimal solution (the shallowest goal node) if multiple goals exist.

Mathematical Reasoning:

- Let $g_1, g_2, \ldots, g_n$ be goal nodes at depths $d_1, d_2, \ldots, d_n$, where $d_1 < d_2 < \cdots < d_n$.
- IDS will find g_1 during the iteration with depth limit $l = d_1$.
- Since IDS does not explore deeper levels until all nodes at the current depth have been explored, g_1 will be found before any deeper goals like $g_2, g_3, \ldots$
- **Conclusion:** IDS is optimal in unweighted graphs because it finds the shallowest goal first, ensuring that the least-cost solution is found.

Optimality in Weighted Graphs:

- **Important Note:** IDS is not optimal in weighted graphs where the cost is not solely determined by depth. To guarantee optimality in such scenarios, a different algorithm like Uniform-Cost Search or A* would be necessary.

Proof of Completeness

Argument for Completeness:

- **Mechanism:** IDS performs a series of Depth-Limited Searches (DLS), starting from a depth limit of 0 and incrementally increasing the depth limit by 1 in each iteration.
- **Process:** In each iteration, IDS explores all nodes at the current depth before increasing the limit. This means that, for a finite state space, IDS will eventually explore all possible depths in the search tree.
- **Guarantee:** Since IDS increments the depth limit with each iteration, it will eventually reach the depth at which the goal node is located, provided that the goal exists within the state space.
- **Handling Infinite State Spaces:** In the case of an infinite state space, if a solution exists at some finite depth ***d***, IDS will find it after ***d*** iterations.

Mathematical Reasoning:

- Let ***d*** be the depth of the shallowest goal node in the search tree.
- IDS will perform a DLS with depth limits l = 0,1,2,…,d
- When the depth limit reaches ***l = d***, IDS will find the goal node because it explores all nodes at that depth.

Conclusion: IDS is complete because it systematically explores deeper levels until the goal is found. If the goal exists at any finite depth, IDS will find it.

IDS Performance Measure

Optimality: IDS is optimal for unweighted graphs, meaning it will always find the shortest path (in terms of the number of edges) to the goal node.

Completeness: IDS is complete, ensuring that if a solution exists, it will be found eventually, even in infinite search spaces.

Time Complexity: The time complexity is $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution. Although this is similar to BFS, IDS performs better in terms of memory usage.

Space Complexity: The space complexity is O(d), where ddd is the maximum depth. This is much more efficient than BFS, which has a space complexity of $O(b^d)$.

IDS Applications

Distributed Systems: In distributed systems, IDS can be used to explore state spaces across multiple nodes or processors while minimizing communication overhead. Each node can handle its own depth-limited search, incrementally sharing results to reach a global solution.

AI Planning: IDS can be applied in automated planning systems where actions can lead to exponentially growing state spaces. By incrementally deepening the search, it can effectively manage the exploration of possible plans and their associated consequences.

Exploration in Robotics: IDS can be utilized in robotic exploration tasks, where robots systematically explore unknown environments. By controlling the depth of exploration, robots can cover areas thoroughly without exceeding memory constraints.

Software Model Checking: In software verification and model checking, IDS can be used to exhaustively explore state spaces of finite-state systems. By incrementally increasing depth, it can verify properties of systems without running out of memory.

Genetic Algorithms: In hybrid approaches that combine genetic algorithms with search techniques, IDS can be used to guide the search process through a population of solutions. This allows for focused exploration of the most promising areas of the solution space while managing computational resources effectively.

Limitations of Iterative Deepening Search (IDS)

1. Redundant Exploration of Nodes

Limitation:

- **Redundant Searches:** IDS repeatedly explores the same nodes at shallower depths during each iteration. For example, nodes at depth 1 are explored in every iteration until the depth limit exceeds 1. This redundancy can lead to inefficiencies, particularly when the search space is large.

Scenarios Where This is Inefficient:

- **Large Search Spaces:** In search problems with a large number of nodes, such as complex combinatorial problems, the repeated exploration of nodes can result in significant overhead, making IDS slower than other algorithms that avoid such redundancy.
- **Problems with Shallow Goals:** In scenarios where the goal is located at a shallow depth, IDS's repeated exploration of shallower nodes adds unnecessary computation, making it less efficient compared to a single BFS.

2. High Computational Overhead in Deep Searches

Limitation:

- **Exponential Time Complexity:** The time complexity of IDS is exponential in the depth of the solution because it revisits nodes multiple times across different depth limits. As the depth of the search increases, the number of nodes revisited grows exponentially.

Scenarios Where This is Inefficient:

- **Deep Solutions:** In search problems where the goal is located at a significant depth, such as deep game trees or long-path puzzles, IDS may become inefficient due to the large number of nodes that need to be revisited across multiple iterations.
- **Complex Planning Problems:** In complex AI planning problems where solutions require exploring deep decision trees, IDS's computational overhead can make it less suitable, especially when compared to more direct depth-first approaches.

3. Memory Usage and Resource Constraints

Limitation:

- **Space Complexity vs. Efficiency:** While IDS is designed to be memory-efficient (similar to DFS), it still requires enough memory to handle repeated depth-limited searches. The memory efficiency can be a trade-off against the time complexity, which may not be ideal in all situations.

Scenarios Where This is Inefficient:

- **Resource-Constrained Environments:** In environments with strict memory or computational constraints, such as embedded systems or mobile devices, the overhead of repeated searches may not be justifiable, leading to inefficiency.
- **Real-Time Applications:** In real-time systems, where quick decision-making is critical (e.g., robotics or online pathfinding), the time required by IDS to revisit nodes might introduce delays, making it less suitable for time-sensitive tasks.

4. Difficulty in Handling Weighted Graphs

Limitation:

- **Not Optimal for Weighted Graphs:** IDS is optimal in unweighted graphs, where the goal is to find the shortest path in terms of the number of edges. However, it does not account for varying edge weights and therefore might not find the least-cost path in weighted graphs.

Scenarios Where This is Inefficient:

- **Weighted Pathfinding:** In scenarios such as road networks or transportation logistics where path costs vary, IDS may explore paths that are longer in terms of cost, even if they are shorter in terms of depth. This makes IDS less effective compared to algorithms like Uniform-Cost Search (UCS) or A*.
- **Cost-Sensitive Problems:** In AI applications where cost considerations are crucial, such as resource allocation or economic modeling, IDS's lack of cost awareness can lead to suboptimal solutions.

5. Inefficiency in Infinite or Very Large Search Spaces

Limitation:

- **Challenges with Infinite Spaces:** IDS is complete in finite search spaces, but in infinite or very large search spaces, it may not be practical. The algorithm might continue searching indefinitely without finding a solution if the goal is located far from the start.

Scenarios Where This is Inefficient:

- **Infinite State Spaces:** In problems with infinite state spaces, such as certain types of puzzles or AI decision-making trees that have no bound, IDS may not terminate in a reasonable time frame, making it impractical for these scenarios.
- **Extremely Large Search Spaces:** Even in finite but extremely large search spaces, IDS may become inefficient as it requires an enormous amount of time to explore deep nodes, especially when the goal is located at a great distance.

6. Lack of Heuristic Guidance

Limitation:

- **Uninformed Search Strategy:** IDS is an uninformed search algorithm, meaning it does not use any domain-specific knowledge or heuristics to guide the search towards the goal more efficiently. It explores all nodes at each depth level without prioritization.

Scenarios Where This is Inefficient:

- **Heuristic-Driven Searches:** In problems where heuristics can provide significant guidance (e.g., in A* search), IDS's lack of direction can make it less efficient, as it does not take advantage of heuristic information to prune the search space.
- **Complex Problem Domains:** In complex domains like AI planning or navigation in large environments, where heuristics can dramatically reduce the search space, IDS may expand many irrelevant nodes, leading to inefficiency.

7. Suboptimal Performance in Wide Search Trees

Limitation:

- **Wide Branching Factors:** IDS performs poorly in search trees with high branching factors, where each node has many children. The algorithm must explore all children at each depth level, leading to an exponential growth in the number of nodes explored.

Scenarios Where This is Inefficient:

- **High Branching Factor Problems:** In problems like large-scale decision-making or certain types of games with many possible moves at each step, IDS can become overwhelmed by the sheer number of nodes, making it less efficient compared to algorithms that can prioritize certain branches.
- **Network Pathfinding:** In network pathfinding scenarios with many alternative routes at each step, IDS may become bogged down by the number of options it must explore, leading to inefficiency.

Uniform Cost Search

Uniform Cost Search (UCS) is a search algorithm that is used to find the least-cost path from a given starting node to a goal node in a weighted graph. It is a variant of Dijkstra's algorithm and is a form of best-first search that expands the node with the lowest path cost first. UCS is particularly useful for finding optimal solutions in scenarios where path costs vary.

Key Characteristics of Uniform Cost Search:

1. **Cost-Based Expansion**: UCS expands nodes based on the cumulative cost to reach them from the start node. It always chooses the node with the lowest total path cost for expansion.
2. **Data Structures**: UCS uses a priority queue (often implemented as a min-heap) to manage the nodes based on their cumulative costs. This allows for efficient retrieval of the next node to expand.

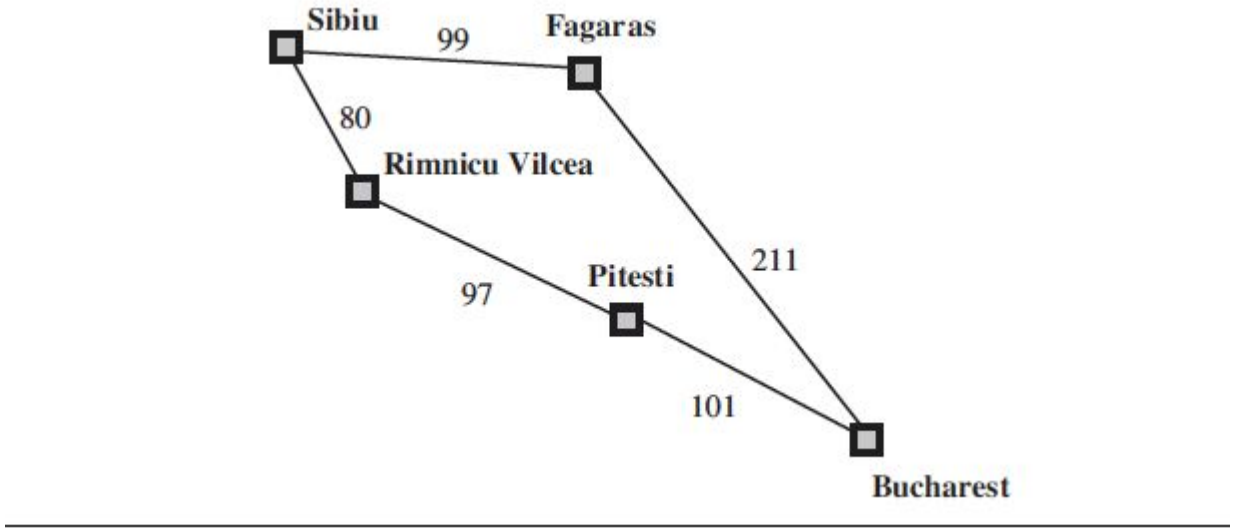
Contrast with BFS and DFS:

- **BFS:** Explores level by level and finds the shortest path in terms of the number of edges (in unweighted graphs).
- **DFS:** Explores deeply before backtracking but does not guarantee finding the shortest or least-cost path.
- **UCS:** Explores paths based on their cumulative cost, ensuring the discovery of the least-cost path in weighted graphs.

Real-world analogy: Imagine you're navigating a city using public transport. You want to reach your destination using the least expensive route, regardless of how many stops it takes. UCS helps you find this optimal route.

Example

Part of the Romania state space, selected to illustrate uniform-cost search.



Dijkstra's Algorithm: A specific case of UCS where all edge costs are non-negative, commonly used in graph-based pathfinding.

UCS Algorithm Steps

Initialize the Priority Queue:

- Create a priority queue (often implemented as a min-heap) to store the frontier. The frontier is the set of all leaf nodes available for expansion at any given point.
- Start by inserting the initial node (start state) into the priority queue with a cumulative cost of 0.

Initialize the Explored Set:

- Create an empty set to keep track of the nodes (states) that have already been expanded (explored) to avoid redundant processing.

Begin the Search Loop:

- **While the priority queue is not empty**, repeat the following steps:

Pop the Node with the Lowest Cost:

- Extract the node with the lowest cumulative cost from the priority queue. This node represents the current path with the smallest cost.
- Let this node be called `current_node`, and its associated cost be `current_cost`.

UCS Algorithm Steps

Goal Test:

- Check if `current_node` is the goal state.
- If `current_node` is the goal state, the algorithm terminates and returns the solution, which includes the path and the total cost to reach the goal.

Add the Current Node to the Explored Set:

- If `current_node` is not the goal state, add it to the explored set to ensure that it is not expanded again.

Expand the Current Node:

- For each child (or successor) of `current_node`:
  - **Calculate the Cumulative Cost:** Determine the cumulative cost to reach the child by adding the edge cost from `current_node` to the child's cumulative cost.
  - **Check if the Child is in the Explored Set:**
    - If the child is not in the explored set or the priority queue, add it to the priority queue with its cumulative cost.
    - If the child is already in the priority queue but with a higher cumulative cost, update the priority queue with the new lower cost.
  - **Note:** UCS prioritizes paths with lower costs, so the priority queue ensures that the node with the lowest cumulative cost is expanded next.

Termination:

- If the priority queue becomes empty and the goal has not been found, this indicates that no solution exists within the given state space, and the algorithm returns failure.

UCS Algorithm

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
    node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
    frontier ← a priority queue ordered by PATH-COST, with node as the only element
    explored ← an empty set
    loop do
        if EMPTY?(frontier) then return failure
        node ← POP(frontier)  /* chooses the lowest-cost node in frontier */
        if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
        add node.STATE to explored
        for each action in problem.ACTIONS(node.STATE) do
            child ← CHILD-NODE(problem, node, action)
            if child.STATE is not in explored or frontier then
                frontier ← INSERT(child, frontier)
            else if child.STATE is in frontier with higher PATH-COST then
                replace that frontier node with child
```

Proof of Optimality

Step 1: Node Expansion and Path Costs

- Consider the moment UCS expands the goal node ***g***. Let ***C(g)*** be the cumulative cost associated with reaching ***g*** along the path UCS followed.
- Assume, for the sake of contradiction, that there exists another path to ***g*** with a lower cost, denoted as ***C'(g) < C(g)***.

Step 2: Contradiction by Assumption

- Since UCS always expands the node with the lowest cumulative cost next, it must have expanded all nodes with cumulative costs less than ***C(g)*** before expanding ***g***.
- If there were a path to ***g*** with a cumulative cost ***C'(g) < C(g)***, UCS would have expanded the nodes along this cheaper path to ***g*** first.
- Therefore, UCS would have reached ***g*** via the cheaper path before considering the path with cost ***C(g)***.

Step 3: Conclusion

- The assumption that there is a cheaper path ***C'(g) < C(g)*** contradicts the way UCS operates, as UCS would have found and expanded this cheaper path first.
- Since UCS expands nodes in order of their cumulative path cost, when UCS expands the goal node ***g***, the path ***C(g)*** must be the minimum possible cost to reach ***g***.
- Therefore, no other path to ***g*** with a cost less than ***C(g)*** exists.

Step 4: Generalization

- This reasoning holds for all nodes in the graph, not just the goal node. Thus, UCS guarantees that when it expands any node, it has found the least-cost path to that node.
- When UCS expands the goal node, it has found the optimal solution to the problem.

Proof of Completeness

Step 1: Initialization and Node Expansion

- UCS starts by initializing the priority queue with the start node ***s*** and a path cost of 0.
- Nodes are expanded in order of increasing cumulative cost, meaning that the node with the lowest cost is expanded first.

Step 2: Handling Non-Negative Costs

- Each edge in the graph has a non-negative cost, meaning that the cumulative cost of any path increases as nodes are expanded.
- Because UCS always expands the node with the smallest cumulative cost, it systematically explores paths in increasing order of their total cost.

Step 3: Exploring All Paths

- Suppose a solution (a path from ***s*** to ***g***) exists.
- UCS will continue expanding nodes and exploring new paths until the goal node ***g*** is reached.
- Because all edge costs are non-negative, the cumulative cost associated with paths cannot decrease, ensuring that UCS doesn't revisit nodes with lower costs unnecessarily.

Step 4: No Infinite Loop or Incomplete Exploration

- Since UCS expands nodes in order of increasing cost, it avoids getting stuck in loops or revisiting nodes excessively.
- If the graph is finite, UCS will eventually exhaust all possible paths leading from sss and will reach ***g*** because it explores every possible path with increasing cost until the goal is found.
- If the graph is infinite but has finite cumulative costs, UCS will still find the goal ***g*** by exploring paths with increasingly higher costs. If a path exists, UCS will eventually expand the goal node.

Step 5: Termination

- UCS terminates as soon as it expands the goal node ***g***. Since UCS prioritizes paths with lower costs, the first time UCS expands ***g***, it does so using the least-cost path.
- Since UCS expands nodes based on cumulative path costs, it ensures that all possible paths to ***g*** are considered, and ***g*** will be reached if reachable.

Conclusion

- **Completeness:** UCS is complete because it systematically explores all possible paths from the start node ***s*** in increasing order of cost. If a path to the goal node ***g*** exists, UCS will eventually find and expand ***g***, ensuring that the goal is reached.

Time Complexity

Assumptions:

- Let ***b*** be the branching factor, which is the average number of children each node has.
- Let ***C**** be the cost of the optimal (least-cost) solution.
- Let ϵ be the smallest positive edge cost in the graph.
- The state space is assumed to be finite, and all edge costs are non-negative.

Basic Operation of UCS

- UCS explores nodes in increasing order of their cumulative path cost.
- Each time a node is expanded, UCS adds its children to the priority queue with their cumulative path costs.
- The priority queue is used to ensure that the node with the lowest cumulative path cost is expanded next.

Number of Nodes Expanded by UCS

- The key to understanding the time complexity is estimating how many nodes UCS might need to expand to find the optimal solution.

Cost Boundaries and Path Costs:

- Let ***C(n)*** be the cumulative cost of the path from the start node ***s*** to a node ***n***.
- For UCS to explore a node ***n*** before finding the goal node ***g***, the cost ***C(n)*** must be less than or equal to ***C**** , where ***C**** is the cost of the optimal solution.
- Since UCS only expands nodes with costs $C(n) \leq C^*$, the number of nodes expanded depends on how many nodes have path costs within this range.

Depth and Branching Factor Consideration:

- The number of nodes that UCS might consider expands exponentially with the depth of the tree. However, in UCS, "depth" is more appropriately measured in terms of path cost rather than the number of edges.
- For each depth level in terms of cost, UCS could potentially explore all nodes whose cumulative costs fall within that level.
- Each node ***n*** can generate up to ***b*** children. If UCS explores nodes up to a cumulative cost ***C****, the number of nodes expanded is related to the total number of nodes within that cost bound.

Estimating the Total Number of Nodes Expanded

- The total number of nodes UCS can expand is determined by how many nodes have a cumulative path cost less than or equal to ***C****.
- Consider the worst-case scenario where UCS might expand nodes even with slightly larger costs ***C(n) > C**** due to ties or small variations in costs. However, in the limit, the number of nodes with path costs within ***C**** is proportional to $b^{C^*/\epsilon}$.
- Therefore, the time complexity of UCS is dominated by the number of nodes with cumulative costs within ***C****, which is approximately:

$$\textbf{Time Complexity} = O(b^{C^*/\epsilon})$$

Conclusion

- The time complexity of UCS is $O(b^{C^*/\epsilon})$, where:
  - ***b*** is the branching factor,
  - ***C**** is the cost of the optimal solution, and
  - ϵ is the smallest positive edge cost in the graph.
- This complexity reflects the exponential growth in the number of nodes UCS must expand as the cost C* increases, relative to the smallest cost increment ϵ.

UCS Performance Measure: Summary

Optimality: UCS is optimal because it expands nodes in order of their cumulative cost, ensuring that the first time it reaches the goal, it does so via the least-cost path.

Completeness: UCS is complete as long as all edge costs are non-negative. It will always find a solution if there is one because it explores all possible paths in order of increasing cost.

Time Complexity: UCS has a time complexity of $O(b^{C^*/\epsilon})$, where b is the branching factor, C^* is the cost of the optimal solution, and ϵ is the smallest positive edge cost. The time complexity grows exponentially with the cost of the optimal path.

Space Complexity: UCS has a space complexity of $O(b^{C^*/\epsilon})$, as it must store all paths in the priority queue. This is similar to its time complexity, meaning UCS can be memory-intensive, especially in large search spaces.

UCS Applications

Logistics and Supply Chain Management: UCS can optimize transportation routes, determining the least-cost path for moving goods from warehouses to distribution centers while considering fluctuating transportation costs.

Telecommunication Network Design: UCS is used to find optimal routing paths for data packets in networks, minimizing latency and costs while maximizing bandwidth and reliability.

AI in Robotics: UCS assists in robotic path planning by navigating complex environments with varying movement costs, allowing robots to conserve energy and maximize efficiency in tasks.

Medical Treatment Pathways: UCS optimizes treatment plans by evaluating different options based on cost-effectiveness and resource allocation, helping identify the most efficient pathway to achieve desired health outcomes.

Urban Planning: UCS helps evaluate construction routes for infrastructure development, allowing urban planners to minimize expenses and environmental impact by prioritizing paths with lower costs.

Limitations of UCS

1. High Memory Usage

Limitation:

- **Space Complexity:** UCS requires significant memory to store all the nodes in the priority queue until the search reaches the goal. Since UCS must store every explored path in the queue to ensure that the least-cost path is expanded first, the space complexity can become problematic, particularly in large or dense graphs.

Scenarios Where This is Inefficient:

- **Large Graphs:** In applications such as mapping large cities or vast networks, where the number of nodes and edges is very high, UCS can consume large amounts of memory, potentially exhausting available resources.
- **Dense Graphs:** In graphs where most nodes are connected by edges, the number of potential paths that UCS needs to store increases rapidly, leading to excessive memory usage.

2. Slow Performance in Graphs with Uniform Costs

Limitation:

- **Time Complexity:** When edge costs are relatively uniform (or identical), UCS behaves similarly to Breadth-First Search (BFS), exploring many unnecessary nodes at the same level of cost. This can lead to slow performance because UCS does not take advantage of cost variations to prune the search space effectively.

Scenarios Where This is Inefficient:

- **Uniformly Weighted Graphs:** In scenarios like grid-based pathfinding where all moves have the same cost, UCS will explore a broad set of nodes, making it slower compared to algorithms that exploit heuristics.
- **Flat Terrain Navigation:** In robotics, if a robot is navigating on a flat surface where all paths are equally costly, UCS might explore many unnecessary paths before finding the goal, leading to inefficiency.

3. Sensitivity to Small Variations in Edge Costs

Limitation:

- **Minimal Edge Cost ϵ\epsilonϵ:** The performance of UCS is sensitive to the smallest edge cost in the graph. If the smallest edge cost ϵ\epsilonϵ is very small, UCS may have to explore a large number of paths before distinguishing between slightly different cumulative costs, resulting in high computational overhead.

Scenarios Where This is Inefficient:

- **Graphs with Minor Cost Differences:** In applications such as financial modeling or logistics planning where small cost differences between paths matter, UCS might expand many similar-cost paths, increasing the computational burden.
- **Network Routing with Variable Costs:** In network routing where data packet delivery costs vary slightly due to congestion or traffic, UCS might inefficiently explore many near-optimal paths, increasing processing time.

4. Inefficiency in Deep Search Spaces

Limitation:

- **Exponential Growth:** UCS's time complexity is exponential in the depth of the least-cost solution, especially in deep search spaces. If the optimal solution is located deep in the search tree, UCS must potentially explore many levels of nodes before finding the goal.

Scenarios Where This is Inefficient:

- **Deep Decision Trees:** In AI decision-making processes, such as game playing where the solution lies deep in the decision tree, UCS can be slow because it needs to explore all possible paths up to a certain depth.
- **Long-Path Planning:** In robotics or planning problems where the optimal solution requires many steps (e.g., complex multi-stage manufacturing processes), UCS might take a long time to reach the goal due to the depth of the search space.

5. Difficulty Handling Negative Edge Costs

Limitation:

- **Incompatibility with Negative Costs:** UCS assumes non-negative edge costs. If the graph contains negative edge costs, UCS can fail to find the optimal solution, as it does not handle negative cycles properly. Negative costs can lead to situations where the cumulative cost of a path keeps decreasing, causing UCS to re-expand nodes and potentially enter an infinite loop.

Scenarios Where This is Inefficient:

- **Economic Modeling:** In economic scenarios where certain paths might have negative costs (e.g., subsidies, rebates), UCS is not suitable because it cannot accurately model and solve such problems.
- **Resource Allocation with Penalties:** In scenarios where penalties (negative costs) apply to certain decisions, UCS may struggle to find the optimal allocation due to its inability to handle negative costs correctly.

6. Inability to Leverage Heuristics

Limitation:

- **Lack of Heuristic Guidance:** UCS is an uninformed search algorithm, meaning it does not use heuristics to guide the search. This can lead to inefficiencies in large search spaces where heuristics could significantly reduce the number of nodes explored by focusing the search on promising areas.

Scenarios Where This is Inefficient:

- **Complex AI Planning:** In applications like AI planning or complex decision-making, where domain-specific knowledge could guide the search towards the goal more efficiently, UCS is less effective because it cannot incorporate this information.
- **Pathfinding in Large, Complex Environments:** In large maps or terrains where the goal is far away, UCS might explore many irrelevant paths that could be avoided with heuristic guidance, making it less efficient than algorithms like A*.

Bidirectional Search

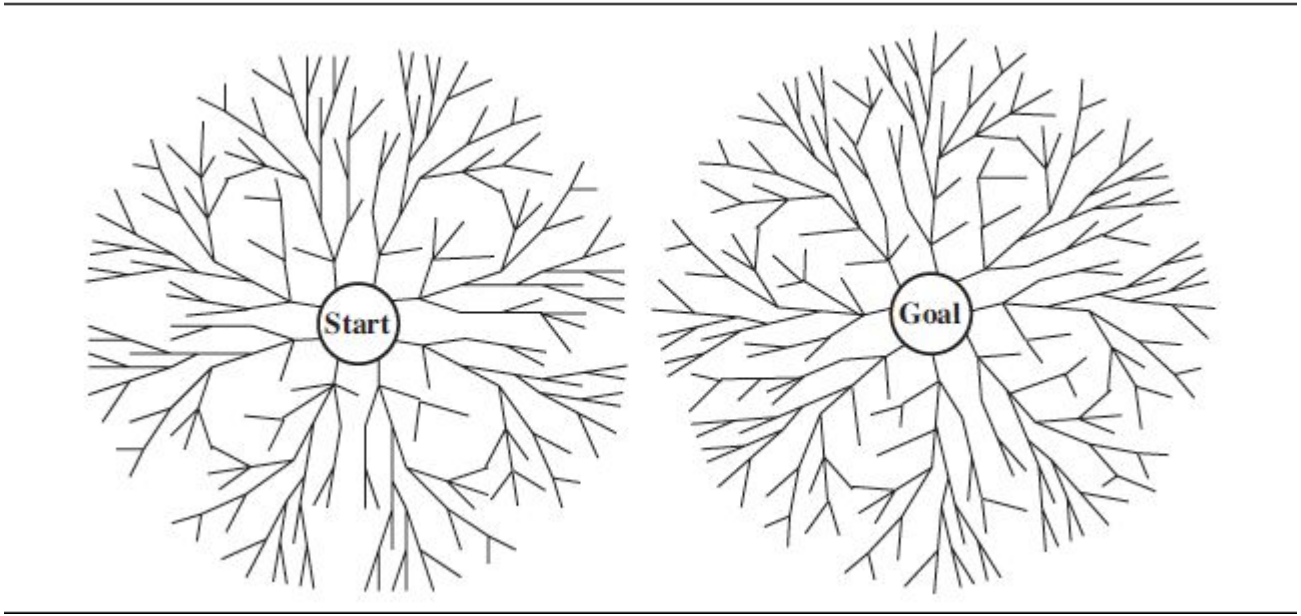
Bidirectional Search is a search algorithm that simultaneously explores the search space from both the initial state (start node) and the goal state, aiming to meet in the middle. This approach can significantly reduce the time complexity of finding a solution compared to unidirectional search methods, particularly in large state spaces.

Key Characteristics of Bidirectional Search:

1. **Two Fronts**: The algorithm maintains two separate searches: one that starts from the initial state and another that starts from the goal state. These searches progress concurrently.
2. **Meeting Point**: The searches continue until they intersect, which indicates that a solution has been found. This meeting point is crucial for determining the optimal path.

The rationale behind Bidirectional Search is that searching from two directions simultaneously can significantly reduce the number of nodes that need to be explored compared to a unidirectional search. In the best case, this approach can cut down the search space to roughly the square root of what would be explored by a single BFS.

Example



A schematic view of a bidirectional search that is about to succeed when a branch from the start node meets a branch from the goal node.

BS Algorithm Steps

- **Initialization:**
  a. Two search fronts are initialized: one starting from the start node and one from the goal node.
  b. Each search front maintains its own queue (like in BFS) or priority queue (in weighted graphs).
  c. Two sets of explored nodes are maintained: one for each search direction.
- **Search Execution:**
  a. In each iteration, alternate between expanding the forward search and the backward search.
  b. Expand the node with the smallest cost (or next in line, in the case of BFS) from each frontier.
  c. For the forward search, explore all unvisited neighbors of the current node and add them to the frontier.
  d. For the backward search, do the same, but considering paths leading to the goal node.
- **Termination:**
  a. The search terminates when a node is found that has been expanded by both the forward and backward searches. This node lies on the shortest path between the start and goal nodes.
  b. The shortest path can be reconstructed by combining the paths from the start node to the meeting point (found by the forward search) and from the goal node to the meeting point (found by the backward search).

BS Algorithm

```
function BIBF-SEARCH(problem_F, f_F, problem_B, f_B) returns a solution node, or failure
  node_F ← NODE(problem_F.INITIAL)                    // Node for a start state
  node_B ← NODE(problem_B.INITIAL)                    // Node for a goal state
  frontier_F ← a priority queue ordered by f_F, with node_F as an element
  frontier_B ← a priority queue ordered by f_B, with node_B as an element
  reached_F ← a lookup table, with one key node_F.STATE and value node_F
  reached_B ← a lookup table, with one key node_B.STATE and value node_B
  solution ← failure
  while not TERMINATED(solution, frontier_F, frontier_B) do
    if f_F(TOP(frontier_F)) < f_B(TOP(frontier_B)) then
      solution ← PROCEED(F, problem_F frontier_F, reached_F, reached_B, solution)
    else solution ← PROCEED(B, problem_B, frontier_B, reached_B, reached_F, solution)
  return solution

function PROCEED(dir, problem, frontier, reached, reached_2, solution) returns a solution
        // Expand node on frontier; check against the other frontier in reached_2.
        // The variable "dir" is the direction: either F for forward or B for backward.
  node ← POP(frontier)
  for each child in EXPAND(problem, node) do
    s ← child.STATE
    if s not in reached or PATH-COST(child) < PATH-COST(reached[s]) then
      reached[s] ← child
      add child to frontier
      if s is in reached_2 then
        solution_2 ← JOIN-NODES(dir, child, reached_2[s]))
        if PATH-COST(solution_2) < PATH-COST(solution) then
          solution ← solution_2
  return solution
```

Time Complexity

1. Understanding the Problem

Let:

- ***b*** be the branching factor (the average number of children per node).
- ***d*** be the depth of the solution (the number of edges in the shortest path from the start node to the goal node).

2. Time Complexity of Unidirectional BFS

In a traditional Breadth-First Search:

- The search expands nodes level by level.
- At depth ***d***, the total number of nodes that could be expanded is the sum of the nodes at all levels from 0 to ***d***.
- The number of nodes at each level ***i*** is approximately b^i.

The total number of nodes expanded by BFS up to depth ***d*** is:

$$\textit{Total nodes} = b^0 + b^1 + b^2 + \cdots + b^d \approx O(b^d)$$

3. Time Complexity of Bidirectional Search

In Bidirectional Search:

- The search is conducted from both the start node and the goal node, meeting somewhere in the middle.
- Instead of searching to depth ***d*** in one direction, Bidirectional Search goes to depth ***d/2*** from the start and ***d/2*** from the goal.

Nodes Expanded by Each Search

For each half of the search:

- The number of nodes expanded by the forward search up to depth ***d/2*** is $b^{d/2}$.
- Similarly, the number of nodes expanded by the backward search up to depth ***d/2*** is $b^{d/2}$.

Since the search is happening in two directions, the total number of nodes expanded by Bidirectional Search is the sum of the nodes expanded in both directions:

$$\textbf{Total nodes in Bidirectional Search} = b^{d/2} + b^{d/2} = 2b^{d/2}$$

4. Comparing the Complexities

- **Unidirectional BFS:** $O(b^d)$
- **Bidirectional Search:** $O(2b^{d/2})$

Since the factor of 2 is a constant, it can be ignored in big-O notation, so the time complexity of Bidirectional Search simplifies to: $\mathbf{O(b^{d/2})}$

Proof of Optimality

To prove the optimality of Bidirectional Search, we will argue that:

1. **Paths Found by Both Searches Are Optimal Up to the Meeting Point:**
   - The forward search finds the shortest path from ***S*** to ***M***.
   - The backward search finds the shortest path from ***G*** to ***M***.
2. **Combination of Paths Forms the Optimal Path:**
   - The path ***S***→***M***→***G*** formed by combining the two optimal paths ***S***→***M*** and ***G***→***M*** is the shortest possible path from ***S*** to ***G***.

Proof of Optimality

Step 1: Forward and Backward Searches Find Optimal Paths

- **Forward Search Optimality:**
  - The forward search from ***S*** expands nodes in order of increasing path length or cost.
  - When the forward search reaches ***M***, it must have found the shortest path from ***S*** to ***M*** because if there were a shorter path, it would have been expanded first.
- **Backward Search Optimality:**
  - Similarly, the backward search from ***G*** expands nodes in order of increasing path length or cost.
  - When the backward search reaches ***M***, it must have found the shortest path from ***G*** to ***M*** because if there were a shorter path, it would have been expanded first.

Step 2: Combining the Two Paths

- **Path from S to G via M:**
  - The path ***S*→*M*** found by the forward search is the shortest path from ***S*** to ***M***.
  - The path ***M*→*G*** found by the backward search is the shortest path from ***G*** to ***M***.
  - Therefore, the combined path ***S*→*M*→*G*** must be the shortest path from ***S*** to ***G*** because it consists of two optimal subpaths.

Step 3: No Shorter Path Exists

- **Suppose there exists a shorter path P′ from S to G that does not pass through M.**
  - Such a path would imply that one of the two searches (either forward or backward) missed a shorter subpath, which contradicts the optimality of the individual searches.
  - Since the forward search finds the shortest path from ***S*** to ***M*** and the backward search finds the shortest path from ***G*** to ***M***, no shorter alternative path ***P′*** can exist.

Conclusion

- **Optimality:** Bidirectional Search is optimal because it finds the shortest path from the start node ***S*** to the goal node ***G***. The two searches meet at a node ***M***, and the combination of the shortest paths from ***S*** to ***M*** and ***G*** to ***M*** forms the overall shortest path from ***S*** to ***G***.
- **No Shorter Path Exists:** If a shorter path existed, it would have been found by one of the two searches before they met, leading to a contradiction. Thus, the path found by Bidirectional Search is guaranteed to be the shortest.

Proof of Completeness

Step 1: Both Searches Explore All Reachable Nodes

- **Forward Search:**
  - The forward search starts from ***S*** and expands nodes level by level (in breadth-first fashion or based on cumulative cost if the graph is weighted).
  - It explores all nodes that can be reached from ***S***, ensuring that no reachable node is skipped.
- **Backward Search:**
  - Similarly, the backward search starts from ***G*** and expands nodes level by level.
  - It explores all nodes that can be reached from ***G***, ensuring that no reachable node is skipped.

Step 2: Searches Must Meet if a Path Exists

- **Assumption of a Path:**
  - Assume that there is at least one path from ***S*** to ***G***. This means there exists a sequence of nodes $S \rightarrow N1 \rightarrow N2 \rightarrow \cdots \rightarrow G$.
- **Meeting of Searches:**
  - As the forward search explores all nodes reachable from ***S*** and the backward search explores all nodes reachable from ***G***, the two searches are guaranteed to meet at some node along the path $S \rightarrow G$.
  - The node where the two searches meet can be any node ***M*** along the path, including ***S***, ***G***, or any intermediate node.

Proof of Completeness

Step 3: Termination at a Solution

- **Meeting Node M:**
  - When the forward and backward searches meet at node ***M***, the forward search has found the path ***S*→*M*** and the backward search has found the path ***G*→*M*** (in reverse).
- **Path Formation:**
  - The algorithm can then combine these two paths to form the full path ***S*→*M*→*G***.
- **Solution Guarantee:**
  - Since both searches are guaranteed to explore all reachable nodes, they will necessarily meet if a path exists. Therefore, a solution is guaranteed to be found.

Handling Edge Cases

- **No Path Exists:**
  - If no path exists between ***S*** and ***G***, one of the searches will eventually exhaust all possible nodes without finding a meeting point. In this case, Bidirectional Search correctly identifies that no solution exists.
- **Graph with a Single Node:**
  - If ***S*=*G***, Bidirectional Search trivially finds the solution without expanding any other nodes.

Conclusion

- **Completeness:** Bidirectional Search is complete because it guarantees finding a path from the start node ***S*** to the goal node ***G*** if such a path exists. The algorithm achieves this by ensuring that both the forward and backward searches explore all reachable nodes from their respective starting points, leading to a meeting point that forms the solution.

BS Performance Measure

Time Complexity:

- **Best Case**: **$O(b^{d/2})$** – In Bidirectional Search, the time complexity is significantly reduced compared to unidirectional search methods like Breadth-First Search (BFS). Since both searches (forward and backward) progress simultaneously and ideally meet in the middle, the depth ddd is effectively halved. Therefore, the time complexity is $O(b^{d/2})$ instead of $O(b^d)$, where b is the branching factor and d is the depth of the solution.
- **Worst Case**: In some cases, if the searches do not intersect early or if one of the search directions encounters a more complex path, the time complexity could approach $O(b^d)$, especially if the intersection happens closer to one end rather than in the middle.

Space Efficiency:$O(b^{d/2})$ – The space complexity of Bidirectional Search is also $O(b^{d/2})$, which is a significant improvement over BFS's $O(b^d)$. This reduction in space complexity occurs because each search only needs to store nodes up to the midpoint of the search space, rather than the entire depth.

Optimality

- **Guaranteed Optimal Solution**:
  - Bidirectional Search guarantees finding an optimal solution if it is implemented with both searches expanding nodes in the same way and if the costs are uniform or if the search algorithm is adapted to handle varying costs (e.g., using a priority queue similar to Uniform Cost Search).
- **Challenges with Non-uniform Costs**: In cases where the edge costs are non-uniform, special care must be taken to ensure that the meeting point between the two searches does not result in a suboptimal solution. This typically involves more complex coordination between the two searches.

Completeness

- **Guaranteed Solution**:
  - Bidirectional Search is complete, meaning it will find a solution if one exists, provided that the search space is finite and fully connected.
- **Handling Disconnected Graphs**: If the search space has disconnected components, Bidirectional Search might fail to find a solution. In such cases, the algorithm needs to ensure that the search directions can indeed meet.

BS Applications

DNA Sequence Alignment: In bioinformatics, Bidirectional Search can be used to align DNA sequences. By starting from both ends of the sequences, the algorithm can efficiently find the optimal alignment that minimizes the difference between the sequences, which is crucial in tasks like genome assembly or comparing genetic data.

Social Network Analysis: In social networks, Bidirectional Search can efficiently find the shortest path between two users. This is useful for determining degrees of separation, finding influencers, or analyzing the spread of information or diseases across a network.

Natural Language Processing (NLP): In NLP tasks like parsing or sentence diagramming, Bidirectional Search can be applied to efficiently match syntactic structures from both the start and end of a sentence, ensuring that grammatical rules are followed and reducing the complexity of processing long sentences.

Reverse Engineering: Bidirectional Search can be used in reverse engineering to match high-level descriptions of a system (like software specifications) with low-level implementations (like code). By searching from both the specification and the implementation, it can efficiently identify where discrepancies or optimizations occur.

Cultural Heritage and Archaeology: In reconstructing ancient texts or artifacts, Bidirectional Search can help match fragmented pieces. By starting the search from both known start points (beginning of a text or structure) and known endpoints (ending), researchers can efficiently piece together incomplete historical data.

Limitations of Bidirectional Search

1. Difficulty in Finding a Meeting Point

Limitation:

- **Synchronization Challenge:** One of the key challenges in Bidirectional Search is ensuring that the two searches (forward and backward) meet. In complex or irregular graphs, it may be difficult to identify a common node where the two searches intersect.

Scenarios Where This is Inefficient:

- **Irregular Graphs:** In graphs with an irregular structure, where nodes are not symmetrically connected, the forward and backward searches might explore vastly different parts of the graph before meeting, leading to inefficiencies.
- **Sparse Graphs:** In sparse graphs, where connections between nodes are limited, the searches may need to cover a large portion of the graph before they intersect, reducing the efficiency gains of bidirectional search.

2. Increased Memory Usage

Limitation:

- **Space Complexity:** While Bidirectional Search reduces the depth of the search, it still requires maintaining two separate frontiers (one for each direction). This can lead to high memory usage, especially in large or dense graphs.

Scenarios Where This is Inefficient:

- **Large Graphs:** In very large graphs, the memory required to store the frontiers from both directions can become substantial, potentially exhausting available resources.
- **Dense Graphs:** In graphs with a high branching factor, the number of nodes added to the frontier at each step can grow rapidly, leading to significant memory consumption as both frontiers expand.

3. Difficulty in Implementing in Weighted Graphs

Limitation:

- **Handling Edge Weights:** In weighted graphs, ensuring that Bidirectional Search remains optimal requires careful management of the priority queues for both searches. The algorithm needs to guarantee that the path cost is minimized when the searches meet, which complicates implementation.

Scenarios Where This is Inefficient:

- **Graphs with Varying Weights:** In graphs where edge weights vary significantly, managing the two priority queues to ensure that both searches proceed optimally can be complex and error-prone.
- **Graphs with Negative Weights:** If the graph contains negative weights, additional mechanisms are required to prevent infinite loops or suboptimal paths, further complicating the implementation.

4. Asymmetry in the Search Space

Limitation:

- **Uneven Exploration:** If the search space is asymmetrical, meaning that the start and goal nodes are not symmetrically located within the graph, one of the searches might expand far more nodes than the other. This uneven exploration can negate the efficiency gains of Bidirectional Search.

Scenarios Where This is Inefficient:

- **Asymmetric Graphs:** In graphs where the start and goal nodes are located in areas with very different branching factors or densities, one search might progress much faster than the other, leading to inefficiencies.
- **Skewed Search Spaces:** In environments like certain game maps or terrain models, where obstacles or barriers create asymmetry, one search might be forced to explore a much larger area, reducing the overall efficiency.

5. Not Suitable for All Graph Types

Limitation:

- **Applicability:** Bidirectional Search is most effective in unweighted graphs or graphs with uniform edge weights. In other types of graphs, particularly those with complex structures or varying costs, other algorithms may be more appropriate.

Scenarios Where This is Inefficient:

- **Graphs with Multiple Goals:** In scenarios where there are multiple goal nodes, Bidirectional Search may need to adjust its strategy or restart multiple times, leading to inefficiencies compared to algorithms specifically designed for multi-goal search problems.
- **Non-Uniform Graphs:** In graphs where edge weights vary widely or are not uniform, Bidirectional Search may struggle to efficiently manage the two searches, leading to inefficiencies compared to heuristic-based searches like A*.

6. Coordination and Management Complexity

Limitation:

- **Algorithmic Complexity:** Coordinating the two searches and ensuring they proceed in a balanced manner can be complex. Managing two sets of data structures, ensuring synchronization, and correctly handling the meeting point adds to the complexity of the implementation.

Scenarios Where This is Inefficient:

- **Real-Time Applications:** In real-time systems, such as robotics or online pathfinding, the overhead of managing two synchronized searches might introduce latency, making Bidirectional Search less suitable.
- **Distributed Systems:** In distributed environments where the search might be spread across multiple machines or processes, coordinating the two searches can introduce significant complexity and overhead, reducing efficiency.

7. Limited Improvement in Small or Simple Graphs

Limitation:

- **Marginal Gains:** In small or simple graphs, where the start and goal nodes are relatively close or the graph structure is straightforward, the benefits of Bidirectional Search may be minimal. The overhead of managing two searches might outweigh the performance gains.

Scenarios Where This is Inefficient:

- **Small Graphs:** In small graphs, the overhead of running two searches may not be justified, as a single BFS or DFS could find the solution with less complexity.
- **Simple Graphs:** In simple, well-connected graphs where paths between nodes are direct and obvious, Bidirectional Search might offer little advantage over traditional unidirectional searches.

Performance Summary

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes[a]	Yes[a,b]	No	No	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]

Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; l is the depth limit.

Superscript caveats are as follows: a complete if b is finite; b complete if step costs ≥ ϵ for positive ϵ; c optimal if step costs are all identical; d if both directions use breadth-first search.

AI Related Inefficiency

Natural Language Processing (NLP)

1.1 Language Parsing and Syntactic Analysis

- **Application:** In NLP, parsing sentences to understand their syntactic structure often involves exploring different possible parse trees.
- **Inefficiencies:**
  - **BFS and DFS:** Both can be inefficient when parsing sentences with ambiguous or complex structures. BFS might explore many irrelevant paths, leading to high memory consumption, while DFS might dive deeply into incorrect parses, wasting computational resources.
  - **DLS:** If the depth limit is set too low, DLS might miss correct parses that require deeper exploration, leading to incomplete or incorrect syntactic analysis.
  - **IDS:** Although it addresses some DFS limitations, IDS's repeated exploration of nodes can be inefficient in languages with long sentences or complex grammars.

1.2 Semantic Search and Question Answering

- **Application:** Semantic search engines and question-answering systems need to explore large knowledge graphs to retrieve the most relevant information.
- **Inefficiencies:**
  - **UCS:** In large knowledge graphs, UCS can become inefficient when the cost (relevance) of connections varies widely. It may explore many low-cost but irrelevant paths, leading to slow response times.
  - **BF:** Bidirectional Search could be inefficient in unstructured knowledge graphs where the start and goal nodes are not symmetrically located, making it hard to find a meeting point.

1.3 Machine Translation

- **Application:** In machine translation, search algorithms are used to explore possible translations by generating and evaluating different sentence structures.
- **Inefficiencies:**
  - **DFS:** DFS might generate long and complex sentence structures that are syntactically valid but semantically incorrect, leading to poor translation quality.
  - **BFS:** BFS could consume excessive memory by exploring all possible translations at each level, making it inefficient for real-time translation tasks.
  - **DLS:** If the translation requires deeper exploration (more complex sentence structures), an inappropriate depth limit in DLS might result in incomplete or inaccurate translations.

Computer Vision (CV)

2.1 Object Recognition and Scene Understanding

- **Application:** Object recognition tasks in CV often involve searching through different possible interpretations of an image to correctly identify objects.
- **Inefficiencies:**
  - **BFS:** In high-resolution images with many possible interpretations, BFS can be memory-intensive and slow, as it explores all possibilities level by level.
  - **DFS:** DFS might explore incorrect object interpretations deeply before backtracking, leading to inefficiencies in real-time applications like autonomous driving.
  - **DLS:** In complex scenes requiring deeper interpretation, DLS might miss correct identifications if the depth limit is insufficient.
  - **IDS:** While mitigating some DFS inefficiencies, IDS's repetitive exploration can still be slow in high-dimensional image spaces.

2.2 Image Segmentation

- **Application:** Image segmentation involves dividing an image into meaningful segments by exploring different pixel groupings.
- **Inefficiencies:**
  - **UCS:** UCS may struggle with varying costs (e.g., edge strength or color differences) between segments, leading to inefficiencies in finding the optimal segmentation.
  - **BF:** In large images, Bidirectional Search might be inefficient if the search spaces from different segments are asymmetrical, leading to imbalanced exploration.

2.3 Pathfinding for Object Tracking

- **Application:** Object tracking in CV involves following an object through a sequence of frames, which requires efficient pathfinding.
- **Inefficiencies:**
  - **BFS:** In complex environments with many possible object trajectories, BFS can quickly consume large amounts of memory, slowing down real-time tracking.
  - **DFS:** DFS might follow incorrect object trajectories deeply before backtracking, leading to poor tracking performance in fast-moving scenes.
  - **DLS:** A poorly chosen depth limit in DLS could result in incomplete tracking if the correct path lies beyond the limit.
  - **IDS:** Repeated node exploration in IDS can slow down tracking, especially in scenarios requiring quick adaptation to changing object movements.

Related Fields

3.1 Robotics (Path Planning and Navigation)

- **Application:** Robots often use search algorithms to plan paths and navigate through environments.
- **Inefficiencies:**
  - **BFS:** In large or cluttered environments, BFS can become memory-intensive and slow, as it explores all possible paths level by level.
  - **DFS:** DFS might lead robots into dead ends or inefficient paths, wasting time and energy.
  - **UCS:** UCS can be inefficient in environments where path costs vary widely, as it may explore many low-cost but suboptimal paths.
  - **DLS:** In complex environments, an incorrect depth limit in DLS could result in incomplete paths, causing the robot to fail in reaching its destination.
  - **IDS:** IDS's repetitive exploration of nodes can be inefficient in dynamic environments where quick pathfinding is necessary.

3.2 Game AI (Decision-Making and Strategy)

- **Application:** Game AI uses search algorithms to make decisions and plan strategies.
- **Inefficiencies:**
  - **BFS:** In games with a large state space, BFS might explore too many irrelevant states, consuming memory and slowing down decision-making.
  - **DFS:** DFS might pursue a poor strategy deeply before recognizing its inefficiency, leading to suboptimal gameplay.
  - **DLS:** DLS might fail to explore deep, effective strategies if the depth limit is set too low, resulting in weak AI performance.
  - **IDS:** The repeated exploration of states in IDS can slow down strategic planning in fast-paced games where decisions need to be made quickly.
  - **BF:** Bidirectional Search can be inefficient in asymmetric game state spaces, where one search direction may progress much faster than the other.

3.3 Automated Theorem Proving

- **Application:** Automated theorem proving involves exploring logical proofs to verify theorems.
- **Inefficiencies:**
  - **DFS:** DFS might explore long, complex proofs deeply before backtracking, leading to inefficiencies in finding shorter, more elegant proofs.
  - **BFS:** BFS could become memory-intensive when exploring large logical spaces, slowing down the proving process.
  - **DLS:** An inappropriate depth limit in DLS might cause it to miss valid proofs that require deeper exploration, leading to incomplete theorem proving.
  - **IDS:** The repeated exploration of proof steps in IDS can lead to inefficiencies in large logical spaces where quick verification is essential.

Informed Search

Search everywhere!!!

Planning and Scheduling

- **Example:** Automated planning for tasks in robotics, logistics, or project management.
- **Search Problem:** The initial state is the starting set of tasks, and the goal state is the completion of all tasks within constraints (time, resources). The search involves finding an optimal or feasible sequence of actions to achieve the goal.

Natural Language Processing (NLP)

- **Example:** Sentence generation, machine translation, or speech recognition.
- **Search Problem:** The initial state is the beginning of a sentence or phrase, and the goal state is the complete, correct sentence or translation. The search explores possible word sequences or grammar structures to form valid sentences.

Optimization Problems

- **Example:** Resource allocation, network design, or maximizing profits in business.
- **Search Problem:** The initial state is the current allocation or configuration, and the goal state is the optimal allocation that maximizes or minimizes a certain objective. The search explores possible configurations to find the optimal one.

Machine Learning Model Training

- **Example:** Hyperparameter tuning or feature selection.
- **Search Problem:** The initial state is the model with default parameters or features, and the goal state is the model with the best performance. The search explores different parameter values or feature combinations to optimize the model's performance.

General Tree Search Paradigm

```
function tree-search(root-node)
        frontier ← successors(root-node)
        while (notempty(frontier))
                { node ← remove-first(frontier)   // lowest f value
                        state ← state(node)
                        if goal-test(state) return solution(node)
                        frontier ← insert-all(successors(node),frontier) }
        return failure
end tree-search
```

General Graph Search Paradigm

```
function tree-search(root-node)
        frontier ← successors(root-node)
        explored ← empty
        while (notempty(frontier))
            { node ← remove-first(frontier)   // lowest f value
                state ← state(node)
                if goal-test(state) return solution(node)
                explored ← insert(node, explored)
                frontier ← insert-all(successors(node),frontier, if node not in explored)
            }
        return failure
end tree-search
```

Best-first Search

- A search strategy is defined by picking the order of node expansion
- Idea: use an evaluation function f(n) for each node
  - Estimate of “desirability”
  - Expand most desirable unexpanded node
- Implementation:
  - Order the nodes in frontier/fringe in decreasing order of desirability

Old Friends

- Breadth First
  - Best First
  - With f(n) = depth(n)
  - BFS considers nodes closer to the start as better, so f(n) could be seen as the depth of the node in the tree. Nodes at shallower depths (closer to the start) are explored first, which means f(n) is lower for these nodes.

- Uniform cost search
  - Best First
  - With f(n) = the sum of edge costs from start to n = g(n)

Three functions

- f(n) = is the evaluation function that best-first search uses to figure out which frontier node to expand
- g(n) = cost from the start node to the current node
- h(n) = guess of the distance/cost from current node to the goal node

f(n) - best node overall cumulatively

g(n) - closer node from the start

h(n) - how far is the goal from me

Informed Search Strategies

Informed search strategies in artificial intelligence (AI) refer to search algorithms that use additional information beyond the basic problem definition to guide the search process towards a solution more efficiently. This additional information typically comes in the form of a heuristic, which is a function that estimates the cost or distance from a given state to the goal state. The purpose of using a heuristic is to make the search process more directed and, ideally, faster by focusing on the most promising paths in the search space.

Key Characteristics of Informed Search Strategies:

1. **Heuristics**: The defining feature of informed search strategies is the use of heuristics. A heuristic is a rule of thumb or an educated guess that helps to estimate how close a current state is to the goal state. For example, in a pathfinding problem, the straight-line distance (Euclidean distance) to the goal can be used as a heuristic.
2. **Efficiency**: By using heuristics, informed search strategies aim to explore fewer nodes than uninformed strategies (like Breadth-First Search or Depth-First Search), making them more efficient in terms of time and memory.

3. **Goal-Directed Search**: Informed search strategies are more goal-directed because they use the heuristic to prioritize which paths to explore first, typically focusing on those that seem most likely to lead to a solution.

4. **Optimality and Completeness**: Some informed search algorithms, are both optimal (they find the best possible solution) and complete (they will find a solution if one exists), provided the heuristic used is admissible (never overestimates the cost to reach the goal).

Heuristics

A **heuristic** is a rule of thumb, a practical method, or a shortcut that helps in decision-making, problem-solving, or discovery. It is not guaranteed to be perfect or optimal, but it is often effective for reaching a satisfactory solution in a reasonable amount of time. In AI, heuristics are used to speed up the process of finding a good solution, especially when the problem space is vast and exhaustive search would be computationally expensive or infeasible.

Characteristics of Heuristics:

1. **Simplification**: Heuristics simplify complex problems by focusing on the most relevant aspects, ignoring less critical details.
2. **Efficiency**: They provide quick, approximate solutions that are "good enough" for the task at hand, even if they are not optimal.
3. **Domain-Specific**: Heuristics are often tailored to specific problems or domains. For example, a heuristic for a chess-playing AI might involve evaluating board positions based on the value of the pieces.
4. **Guidance**: In search algorithms, heuristics guide the search process by estimating which paths are more promising, reducing the number of states that need to be explored.

Heuristics Function

A **heuristic function** (often denoted as `h(n)`) is a specific type of heuristic used in search algorithms. It is a function that estimates the cost or distance from a given node (or state) `n` in the search space to the goal state. The heuristic function helps to prioritize which nodes should be explored next based on their estimated proximity to the goal.

Properties of Heuristic Functions:

1. **Admissibility**:
   - A heuristic function is **admissible** if it never overestimates the true cost to reach the goal from any node. In other words, `h(n)` is always less than or equal to the actual lowest cost to reach the goal.
   - Admissible heuristics are crucial in ensuring that search algorithms like A* find the optimal solution.
2. **Consistency (or Monotonicity)**:
   - A heuristic function is **consistent** if, for every node `n` and every successor `n'` of `n`, the estimated cost of reaching the goal from `n` is no greater than the cost of reaching `n'` plus the estimated cost of reaching the goal from `n'`. Mathematically, this means: $h(n) \leq c(n,n') + h(n')$ where `c(n, n')` is the actual cost of moving from node `n` to node `n'`.
   - Consistency ensures that the heuristic function never decreases along a path, which helps in simplifying the implementation of search algorithms.
3. **Informedness**:
   - The **informedness** of a heuristic function refers to how accurately it estimates the true cost to the goal. A more informed heuristic provides estimates closer to the actual costs, making the search more efficient by reducing unnecessary exploration of non-promising paths.

Examples of Heuristic Functions

Manhattan Distance: In grid-based pathfinding problems, such as navigating through a maze, the Manhattan distance (sum of the absolute differences in the x and y coordinates) between the current node and the goal is a common heuristic. It is admissible and often used in algorithms like A*.

$$h(n) = |x_2 - x_1| + |y_2 - y_1|$$

Euclidean Distance: In problems where the cost to move from one point to another is based on straight-line (as-the-crow-flies) distance, the Euclidean distance (the direct distance between two points) can be used as a heuristic.

$$h(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Examples of Effective Heuristics

Pathfinding in Grid-Based Environments

- **Heuristic:** Manhattan Distance
- **Why It's Effective:** The Manhattan distance provides a heuristic for grid-based movement where diagonal moves are not allowed. It is simple to compute and provides a good estimate of the actual path cost.

Sliding Puzzle (8-puzzle, 15-puzzle)

- **Heuristic:** Number of Misplaced Tiles or Manhattan Distance
- **Why It's Effective:** Both heuristics provide estimates for the minimum number of moves required to solve the puzzle. The number of misplaced tiles heuristic is simple, while the Manhattan distance gives a more refined estimate by accounting for how far each tile is from its goal position.

Traveling Salesman Problem (TSP)

- **Heuristic:** Minimum Spanning Tree (MST)
- **Why It's Effective:** The MST heuristic estimates the lower bound on the cost of completing the tour by connecting all cities without returning to the start.

Real-World Examples of Heuristics

Navigational Heuristics (GPS and Mapping Applications):

- **Example:** When using a GPS to find the shortest route from your current location to a destination, the system might use the straight-line (Euclidean) distance between your location and the destination as a heuristic.
- **Purpose:** This heuristic helps the GPS prioritize routes that move closer to the destination, even though the actual travel path might involve turns and detours.

Medical Diagnosis:

- **Example:** A doctor might use heuristics when diagnosing a patient based on symptoms. For instance, if a patient has a sore throat, fever, and swollen lymph nodes, the doctor might quickly lean towards diagnosing strep throat based on these common indicators.
- **Purpose:** This heuristic allows the doctor to make a preliminary diagnosis quickly, which can then be confirmed with further testing.

Online Search Algorithms:

- **Example:** Search engines like Google use heuristics to rank web pages. For instance, pages that contain the exact keywords being searched for are ranked higher because they are more likely to be relevant.
- **Purpose:** This heuristic helps the search engine return the most relevant results quickly, improving the user experience.

Shopping and Decision-Making:

- **Example:** When shopping online, a heuristic might be to choose a product with the highest number of positive reviews. While not a guarantee of quality, this heuristic simplifies the decision-making process by using crowd wisdom.
- **Purpose:** This approach helps consumers make quicker decisions without analyzing every available option in detail.

Heuristics in Chess (Game AI):

- **Example:** In chess, a common heuristic is to control the center of the board. While not a strict rule, it's generally advantageous to control the center, leading to better mobility and control over the game.
- **Purpose:** This heuristic helps players (and AI) make strategic decisions more quickly by focusing on a key aspect of gameplay rather than calculating every possible move.

Software Development (Rule of Thumb for Bug Fixes):

- **Example:** A common heuristic in software development is that the earlier a bug is found and fixed, the cheaper it is to address. Therefore, developers prioritize writing tests and catching bugs during the development phase rather than after deployment.
- **Purpose:** This heuristic helps in managing resources effectively by addressing issues at a stage where they are easier and less costly to fix.

Types of Informed Search Strategies

1. Greedy Best-First Search
2. A* Search
3. Bidirectional A* Search
4. Iterative Deepening A* Search
5. Memory-Bounded A* Search
6. Weighted A* Search
7. Recursive Best-First Search
8. Beam Search

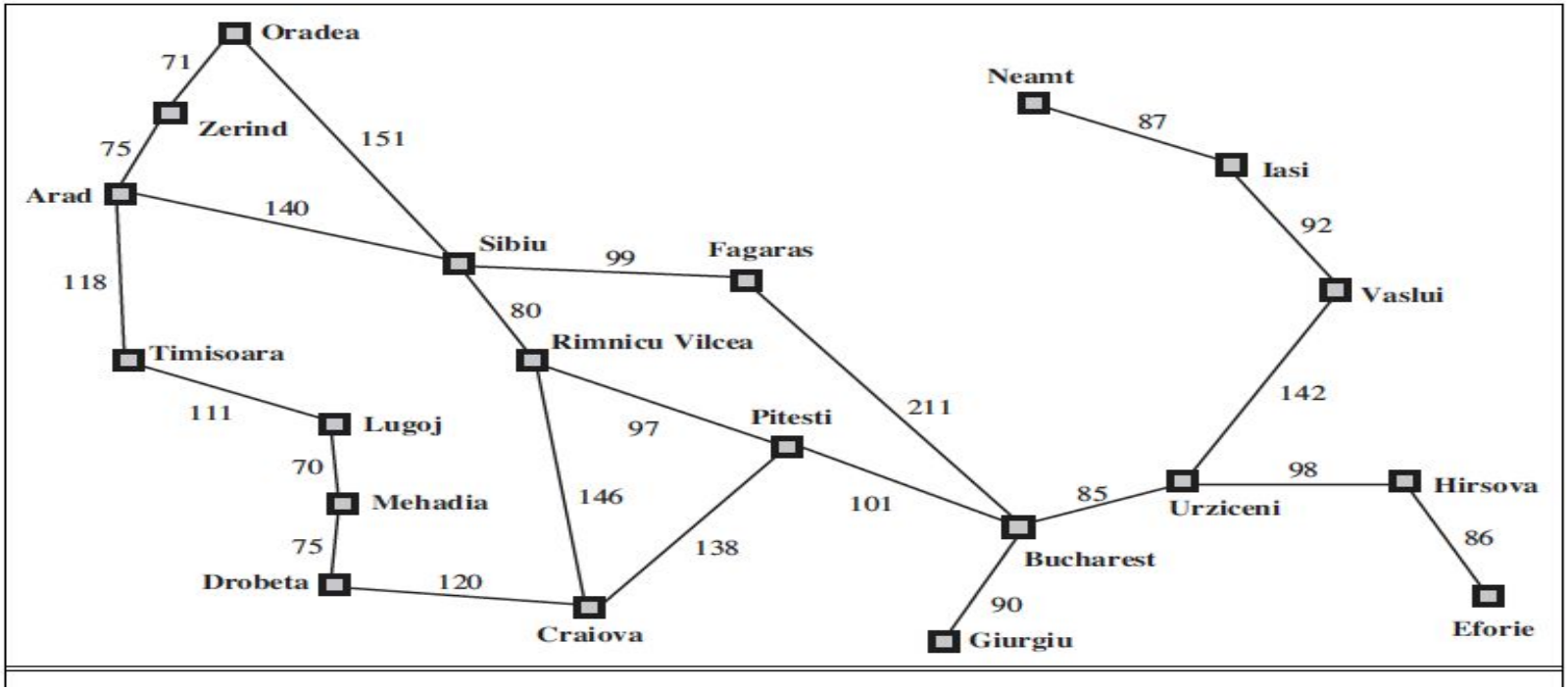
Greedy Best-First Search

Greedy Best-First Search (GBFS) is an informed search algorithm that aims to find the most promising path to a goal by expanding the node that appears closest to the goal based solely on a heuristic function. Unlike other search algorithms that consider both the cost to reach a node and the estimated cost to reach the goal, Greedy Best-First Search focuses only on the heuristic estimate, making it "greedy" in its approach.

Here, f(n) = h(n).

How Greedy Best-First Search Works

1. **Initialization**:
   - Start with an initial node, usually the starting state of the problem.
   - Add this node to a priority queue (often implemented as a min-heap) where nodes are prioritized based on their heuristic value `h(n)`.
2. **Node Expansion**:
   - Extract the node with the lowest heuristic value from the priority queue (this is the "greedy" part, as it selects the node that seems closest to the goal).
   - If this node is the goal, the search ends successfully.
   - If not, expand the node by generating all its successors (i.e., possible states that can be reached from the current state).
3. **Adding Successors to the Queue**:
   - For each successor, compute its heuristic value `h(n)` and add it to the priority queue.
   - The priority queue is then updated to ensure that the node with the lowest `h(n)` will be expanded next.
4. **Repeat**:
   - Continue expanding nodes by repeatedly extracting the node with the lowest `h(n)` and adding its successors to the queue until the goal is found or the queue is empty (indicating that no solution exists).



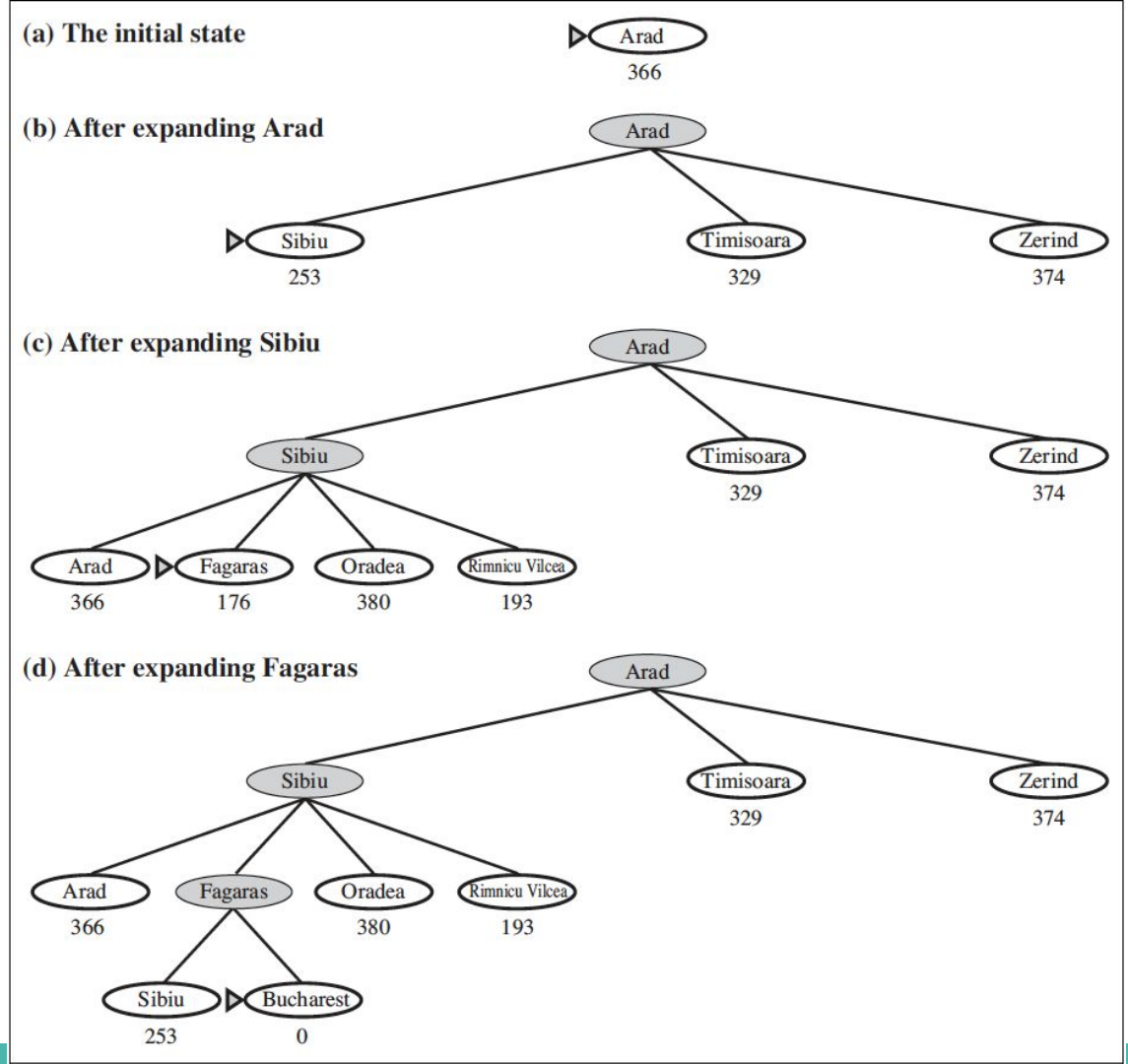
A simplified road map of part of Romania

Values of h_{SLD}—straight-line distances to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Notice that the values of h_{SLD} cannot be computed from the problem description itself. Moreover, it takes a certain amount of experience to know that h_{SLD} is correlated with actual road distances and is, therefore, a useful heuristic.

Stages in a greedy best-first tree search for Bucharest with the straight-line distance heuristic h_{SLD}. Nodes are labeled with their h-values.



Properties of Greedy Best-First Search

1. **Heuristic-Driven**: GBFS relies entirely on the heuristic function `h(n)` to make decisions. It does not consider the actual cost to reach a node, making it greedy.
2. **Not Optimal**: Since GBFS only considers the heuristic and ignores the actual cost to reach nodes (`g(n)`), it is not guaranteed to find the optimal solution. It may choose a path that looks promising (based on the heuristic) but turns out to be suboptimal.
3. **Not Complete**: GBFS is not complete, meaning it may fail to find a solution even if one exists, especially if the heuristic misguides the search into dead ends or cycles in case of tree-search paradigm, but complete in case of graph-search paradigm.
4. **Efficiency**: GBFS can be faster than algorithms like A* because it often expands fewer nodes. However, this speed comes at the cost of potentially finding suboptimal or incomplete solutions.
5. **Memory Usage**: GBFS stores all generated nodes in memory, like A*. However, because it doesn't keep track of the actual cost `g(n)`, it might require less memory than A* in some cases.

Advantages of Greedy Best-First Search

- **Speed**: GBFS can be very fast, especially in problems where the heuristic is well-chosen and closely approximates the true cost to the goal.
- **Simplicity**: The algorithm is relatively simple to implement, focusing solely on the heuristic to guide the search.

Limitations of Greedy Best-First Search

1. Suboptimal Solutions

- **Description:** Greedy Best-First Search is not guaranteed to find the shortest or optimal path to the goal. It focuses solely on the heuristic value, ignoring the actual cost to reach a node from the start.
- **Example in AI:** In a pathfinding AI, GBFS might find a path that looks promising based on the heuristic (e.g., straight-line distance), but it could turn out to be much longer than the optimal path due to obstacles or higher costs not accounted for by the heuristic.

2. Susceptibility to Local Minima

- **Description:** GBFS can get "stuck" in local minima, where it chooses a path that seems best based on the heuristic but is not actually the best overall path. This happens because it doesn’t backtrack or explore alternative paths once a promising-looking route is chosen.
- **Example in AI:** In game AI, GBFS might choose a series of moves that seem to bring the player closer to a win (based on a heuristic), but the strategy leads to a dead end or a poor overall position because it doesn't consider the broader game context.

3. Heuristic Dependence

- **Description:** The effectiveness of GBFS is highly dependent on the quality of the heuristic. If the heuristic is poorly chosen or misleading, the algorithm's performance can degrade significantly, leading to inefficient searches or incorrect solutions.
- **Example in AI:** In natural language processing (NLP) tasks like sentence generation, a heuristic that poorly estimates the likelihood of a phrase fitting into a sentence context can lead to grammatically incorrect or nonsensical outputs.

4. Incomplete Search

- **Description:** GBFS is incomplete in certain cases, meaning it may fail to find a solution even if one exists. This can occur in large or infinite search spaces where the algorithm might get stuck in a loop or continue exploring suboptimal paths indefinitely.
- **Example in AI:** In complex planning tasks like robot navigation, GBFS might endlessly explore a series of moves that seem promising but fail to reach the destination because it doesn’t explore alternative routes that may be longer initially but lead to the goal more effectively.

5. Lack of Global Awareness

- **Description:** GBFS lacks global awareness since it only considers the immediate heuristic value without accounting for the overall structure of the problem. This can lead to inefficient exploration of the search space, especially in problems where the solution requires understanding the broader context or making trade-offs.
- **Example in AI:** In search-based problem-solving like puzzle-solving (e.g., Sudoku), GBFS might focus on filling one row or column without considering how this will affect the overall solution, potentially leading to conflicts later that are hard to resolve.